

Canonical Junior Ubuntu Software Engineering Role

Written interview and psychometric assessment

Engineering experience:

- **Describe a skill or knowledge you acquired recently that has been impactful for you. Why did you make this investment? What has the outcome been?**

Recently, I revisited the fundamentals of Dart and Flutter with a focus on architectural decision-making. I explored global error handling strategies and compared state management approaches such as BLoC and Provider to better understand their trade-offs and appropriate use cases.

Although I had been building features in a Flutter application for over a year, I realized much of the core architecture had been designed by senior engineers. While I understood how it worked, I wanted to strengthen my ability to design such systems independently.

To address this, I systematically reviewed Dart fundamentals and analyzed parts of our codebase I had not originally built, breaking down architectural patterns and abstractions to understand the reasoning behind them.

As a result, I now approach new features with stronger architectural judgment and with more confidence in designing maintainable systems from the scratch.

- **What new skill would you like to learn? Why do you think this is important or timely or interesting? Why do you think you will be good at it?**

I would like to deepen my understanding of how operating system-level tools and interfaces are built, particularly GUI systems and internal development tools. Rather than only using IDEs, file explorer or installers, I want to understand how they are designed, rendered and distributed – from UI architecture to packaging and delivery through a package manager.

I am especially interested in how Flutter is being used within Ubuntu, such as the modern Ubuntu installer. Seeing Flutter applied beyond mobile and web into system-level interfaces – along with libraries like Yaru.dart that adapt Flutter to Ubuntu's design language – is particularly exciting to me.

I believe this is timely and important because Flutter is expanding beyond traditional platforms, and Ubuntu is actively investing in it for desktop experiences as well as for cross-platform apps. Understanding how cross-platform UI framework integrate with operating systems would strengthen both my frontend expertise and my systems-level thinking.

I think I would be good at this because Flutter is already my primary frontend framework. I am comfortable reading documentation and exploring codebases, and I genuinely enjoy understanding how things work internally. My background across multiple stacks has taught me that once core architectural principles are understood, adapting to new layers of abstractions becomes much easier.

- **What kinds of software projects have you worked on before? Which operating systems, development environments, languages, databases?**

I have worked on professional and personal projects ranging from mobile applications (Flutter) and web systems (Angular / React) to Interactive experiences (AR / Games with Unity).

Operating system: Windows and Ubuntu

Languages: Dart, C#, Java, SQL, Python (working familiarity)

Front end: Flutter (Primary), Angular & React (working familiarity)

Back end: Java Springboot (Primary), .NET & Node (Project Experience)

Databases: PostgreSQL & SQLite (Primary), MongoDB & SQL Server (Project Experience)

Development Environments: Unity Engine, Android Studio, VSCode, IntelliJ, Visual Studio

I began my development journey with game development in Unity, where I built a strong foundation in C# and object-oriented programming. The experience made it easier to transition into .NET and later into backend and full stack systems.

Across different projects and tech stacks, I found that core principles – such as clean architecture (i.e. MVC/MVVM), API communication and data handling remain consistent.

Working across different environments has reinforced my belief that technology stack is merely a tool, while problem-solving, system design and clean code patterns truly matter and this allows me to quickly learn new tools and deliver reliable solutions.

- **Would you describe yourself as a high-quality coder? Why?**

Yes, I consider myself a high-quality coder, while recognizing that quality is something I continuously improve.

Before writing code, I focus on fully understanding the problem and then breaking it into manageable parts. I usually start with a simple working solution then refine it to improve correctness, readability and efficiency. I also review my work carefully to catch edge cases and potential bugs.

I follow consistent formatting and naming conventions and aim to keep code modular and reusable. Regardless of the task size, I write code with long-term maintainability in mind knowing it may need to be understood or modified days, months or even years later.

I also try to make things easier for future contributors by leaving clear comments or TODO notes in places where code satisfies current requirements but may require adjustments later. This approach has been recognised positively by my teammates, particularly those who work on modules I previously worked on. These notes are helpful when enhancements come from other teams or when a quick fix is required for a critical issue.

Debugging is another area of strength I believe that contributes to code quality. My debugging skills were already solid before joining my current role but now they have become much sharper with experience. I am often among the first people called when an issue occurs from both my team and other teams. Although I am responsible for a few specific

components, I am frequently utilized as a generalist when problems arise elsewhere in the system just like I am WD-40 Multi purpose product in a toolbox – because when something stops working with unexpected reason, I can help moving things again by either solving it by myself or even finding root cause.

My tasks often begin with minimal information such as “Module A is not displaying”, “Wrong results are shown” or “File is not visible”. From there, I investigate the issue, identify the root cause and then either fix by myself or report it to the responsible team. While working on issues, I also frequently identify and report other bugs or potential issues in the codebases by reading or based on understanding which helps identifying an issue even before QA testing.

For me, high-quality code comes from discipline, attention to detail and learning from experience and because of this I am always evolving.

- **Describe your experience building large systems with many services - web front ends, REST APIs, data stores, event processing and other kinds of integration between components. What are the key things to think about in regard to architecture, maintainability, and reliability in these large systems?**

I have been working for about year and a half on a healthcare application used by frontline health workers, often in areas where connectivity is unreliable. Because of this constraint, the system follows an offline-first design. Data is stored locally on the device and the synchronized with the server when connectivity becomes available.

Working on this system presented several practical challenges that appear in these large systems. For example, synchronization is not simple data transfer between client and server. We had to handle failed upload / download, partial transmitted files and also redundant data. This required cancel and retry mechanism for API calls, file validation, making data / query logic which works for two different databases SQLite and postgresQL (Especially for different versions of SQLite per Android version) and those edge cases of server getting redundant data.

Another important aspect of the system behaviour is that it is driven by JSON configuration. The mobile client reads configuration that determines UI behaviour and workflows. A separate web-based configuration tool allows admins to update the configurations. This reduces requirement of distributing update release build and deployment friction.

From this experience if I were to list key things about large system development –

First, **clear module separation** – as this helps with system growth. It becomes easy to debug issues or have unintended side effects when new features are being developed.

Second, **strong error handling and logging** – with large complex systems number of things which can go wrong increases exponentially. Error handling and logging is way to gracefully deal with failure.

Third, **wrapper and abstraction** – for boiler plate code reduction, so same code for each similar feature or implementation can be quick, easy to maintain and readable.

Fourth, **appropriate API design** – some API can be basic JSON data fetch while some can be uploading complex data like byte stream for a file. Not all require same complex config like timeout, cancel logic.

- **Outline your thoughts on quality in software development. What practices are most effective to drive improvements in quality?**

For me, quality in software development means writing software that is reliable in production environment, easy to maintain as system evolves and easy to understand for debugging any issues. This is achieved by planning from design phase and applying appropriate practices throughout development cycle.

Proper planning and designing should be done whether it is schedule and timeline or approaches for addressing the problem. It increases problem clarity and reduces effort.

Consistency and code clarity should be practiced as code is typically read far more often and it needs to be understandable. Consistent clear code reduces effort to translate intent and makes debugging or enhancement easy. Coding standard helps team to be on same page for name, spacing and format and have uniform codebase.

Logging and Error handling is necessity for software development. As failure occurs in any way and if not dealt, they ruin user experience or even make software unreliable.

Documentation should be done with development as it helps developers convey why something was done and also proper documentation with README work well for onboarding.

Testing should be conducted without fail. Test cases should be clear even before starting development and all cases should be considered during planning which helps with clarity in development also.

- **Outline your thoughts on documentation in large software projects. What practices should teams follow? What are great examples of open-source docs?**

Documentation in large software projects is a core communication layer between developers, teams and users. It explains not only how a system works, but why certain architectural or design decisions were made. In multi-team environments or long lived / large systems documentation reduces onboarding time, prevents knowledge silo and makes future changes safer and increasing user friendliness.

In large projects, documentation should evolve along with the codebase. It is most effective when it is updated as features are implemented rather than written later. Developer-focused documentation should live close to the code, explaining reasoning and trade-offs rather than stating what the code does (As it can be achieved by just reading the code). Repository-level documentation – such as architecture overview, module responsibilities and integration instructions should be easily accessible and even version controlled.

For onboarding, a well-structured README with installation, system overview and clear quick start path is essential. Having tutorial (or it's link) and perhaps diagram / screenshots for complex system is helpful and valuable.

Strong open-source examples include [Flutter](#), [Angular](#) and [Ubuntu](#). Flutter and angular documentation balance beginner-friendly tutorials with deeper conceptual explanations and API references, making them useful at multiple skill levels. Ubuntu's documentation provides both user-facing guides and technical references for system configuration, package management and troubleshooting. This serves different types of users effectively. Package managing platforms like [pub.dev](#) and [npm](#) also demonstrate value of README-driven documentation, changelogs and versioned release notes for libraries.

So ultimately, good documentation is structured, searchable and continuously maintained (like code quality it is not one time effort but ongoing discipline) which supports collaboration, scalability and long termed sustainability of a project.

- **Outline your thoughts on user experience, usability and design in software. How do you lead teams to deliver outstanding user experience?**

User experience and usability begin with empathy – to think how would I want the software if I were the layman end user with no excess information about it. Software should not only function correctly but also should feel intuitive and predictable for the person using it. Good design communicates intent clearly without friction and supports user for completing task efficiently.

If leading a team, I start by clarifying requirements and identifying the core user problem. If requirements are ambiguous, I try to model the system from the perspective of the end user and define what an ideal interaction would look like. From there, I translate that vision into simple wireframe or design sketch to align the team before starting development.

Once the user flow is clear, I evaluate technical feasibility. For delivering strong UX, balancing ideal ambition with practical constraints such as time, resources and architectural complexity is needed. I break the work in features which are must-have and nice-to-have. I then estimate effort and prioritize feature development timeline. This ensures the core experience is polished and enhancements have a schedule and planning even after release.

An often-overlooked part of user experience is how the system behaves under failure conditions. Network issues, delays, partial data, or unexpected errors can significantly impact usability. I believe teams should design for these scenarios explicitly — providing clear feedback, meaningful error messages, and graceful recovery mechanisms. Reliability and transparency directly influence user trust.

Throughout development, I believe iteration and feedback are critical. Even in academic team settings, I found that treating stakeholders (Professor and department head at that time) as clients and refining based on their input led to better outcomes. Clear communication, shared visual references and structured prioritization were key to delivering a coherent user experience.

- **Outline your thoughts on performance in software engineering. How do you ensure that your product is fast?**

Performance in software engineering starts at the design stage. A fast product is rarely the result of late optimizations; it is the outcome of thoughtful architectural decisions. This includes selecting appropriate data structures and algorithms, minimizing unnecessary

network calls and database queries, introducing caching where it adds value, and using techniques such as lazy loading for heavy resources. It also means preventing redundant rendering, duplicate backend processes, and blocking operations on the main thread.

Relying on profiling tools to guide optimization rather than guessing is always a smart move. Tools such as those in Android Studio, browser developer tools, and Unity's performance statistics help identify bottlenecks and areas that require refactoring. Testing on real devices and in real-world environments is equally important, as performance issues often surface under unreliable networks or constrained hardware conditions.

While working on a healthcare application used in areas with unstable connectivity, I encountered a vaccination module that required validating multiple conditions before recommending a vaccine. One key factor was location, where the hierarchy could be large and needed to be evaluated against a substantial vaccine dataset. To improve performance, the location hierarchy was cached per patient, reducing repeated computation across vaccine checks. JSON parsing results were also cached to avoid redundant processing, and recursive retrieval was used to minimize database fetch operations. These changes significantly reduced processing time while keeping the logic maintainable.

At the same time, optimization should be intentional and justified. Optimizing too early or without clear need can reduce readability and increase system complexity. Performance must be balanced with clean, maintainable code.

To ensure a product is fast, performance considerations are embedded into system design, clear expectations are defined, results are measured, and improvements are made where they provide meaningful impact.

- **Outline your thoughts on security in software engineering. How do you lead your engineers to improve their security posture and awareness?**

Security in software engineering should be considered from the beginning of system design, not added as an afterthought. I believe in following secure-by-design principles, ensuring that authentication, authorization, data protection, and communication security are properly implemented from the beginning.

In practice, I have worked with token-based authentication (JWT) and implemented OAuth login with Google. I have also built an OTP-based forgot and reset password flow. I understand the importance of using HTTPS for encrypted communication, not exposing sensitive data in URLs or logs, hashing passwords before storing them in the database, and using role-based access control to restrict what different users can access.

Improving a team's security posture requires awareness and technical practices. One way to do this is integration of security checks in development. Code reviews also can focus on security considerations like input validation or hard code secrets.

Latest dependencies can have fixes for known vulnerabilities. I was aware of this but it was reminded again deeply to me when there was a supply chain attack in September 2025 which compromised npm packages. It did not affect my work personally but since I worked with a React-based team closely, I got a learning experience from their discussions.

This brings another practice which can be used in increasing awareness of engineers – asking them to think about potential risks like ‘what if I were put a SQL injection here will our system hold or it will execute the query?’

My experience is mostly hands-on, and I see security as an area where I want to keep improving.

- **Outline your thoughts on devops and devsecops. Which practices are effective, and which are overrated?**

DevOps focuses on improving collaboration between development and operations, mainly through automation and shared responsibility. The goal is to make software delivery more reliable and less error-prone.

DevSecOps extends this idea by making security part of the development process rather than something addressed at the end.

In my current role, I primarily work on development tasks. We use merge requests and code reviews, which I see as important collaborative practices. However, deployments are handled manually, so I have not yet had hands-on experience with CI/CD pipelines or automated infrastructure.

For effective practises, automated testing, monitoring and logging can be considered effective and adopting complex infrastructure without need or too much obsession to tools maybe counterproductive.

So, in short, I would like to say that I am familiar with concept of devops, but I would like to gain deeper knowledge to grow in this area. Especially since I’m particularly interested in learning how these processes are managed in Linux environments like Ubuntu.

Domain Specific Experience:

- **Outline your thoughts on open source software development. What is important to get right in open source projects? What open source projects have you worked on? Have you been an open-source maintainer, on which projects, and what was your role?**

Open-source software development plays an important role in engineering as it enables collaboration, transparency and shared innovation. Source code is public therefore projects can benefit from feedback and contribution from the community. This often leads to faster improvements and better security with many eyes looking the very code of the project.

For open-source projects several things are needed for right approach. This can include clear and easily accessible documentation for new users and contributors understanding. Documentation should describe both how project works and how to participate. The guidelines, issue tracking and review are key for managing open-source project as they help collaboration.

Another aspect can be maintenance. Maintainers will have a responsibility to ensure governance – or in other words ensure that contributions follow design and coding principles and long-term goals. Some automation can also be implemented for ease.

In my current role, I work with a platform that is based on an open-source codebase distributed under the Eclipse Public Licence 2.0 and hosted on GitHub. The project began as a fully open-source platform and my company maintains it while provide enterprise grade support, configuration and implementation services for clients. My role primarily focuses on implementing and configuring system for real-world deployments rather than contributing directly to the upstream codebase.

Although I have not yet been a maintainer or direct contributor to an open-source project, working closely with an open-source based program has given me an appreciation for how such system evolve and maintained. Especially the exposure to the challenges like managing configs, delivering reliable release for different implementations. This is why I am particularly interested in becoming more involved in open-source development in the future and sharpening my skills.

- **How comprehensive would you say your knowledge of a Linux distribution is, from the kernel up? How familiar are you with low-level system architecture, runtimes and Linux distro packaging? How have you gained this knowledge?**

My knowledge of Linux distributions has developed gradually through both academic exposure and practical experience. My first introduction to Linux was during my school studies, where I was introduced to Ubuntu. Later, during university, I learned more about operating system fundamentals such as process management, file systems, and basic system architecture.

My interest in Linux grew further when I began exploring alternatives while troubleshooting performance and reliability issues on my personal laptop. During that process, I started researching different Linux distributions and experimented with installing systems such as Ubuntu. Because I had already worked with Linux concepts during university, I was able to

become comfortable with the environment, including command-line usage, system configuration, and package management.

Currently, my work laptop also runs Ubuntu as the primary operating system, which allows me to use Linux on a daily basis for development work. Regular usage has made me comfortable with tools and environments such as package management using *apt*, *snap*, and *dpkg*, command-line utilities like *ssh* and *bash*, and system tools such as *grep*, *top*, and file permission management with *chmod*. I also frequently troubleshoot configuration or environment issues while working in this environment.

Beyond day-to-day usage, I have a conceptual understanding of the role of the Linux kernel, system services, and process management within a Linux system. I am also familiar with the general structure of Linux distributions and how package managers handle software installation and dependency management.

While I am comfortable working with Linux systems in development environments, I continue to deepen my understanding of Linux internals, low-level system architecture, and distribution tooling through hands-on exploration and continued learning.

- **Detail your experience and contributions to any Linux distributions. You might include details of any work in Ubuntu, Debian, Fedora, NixOS, Arch Linux, etc. Feel free to include links to any notable contributions.**

I have not yet contributed directly to a Linux distribution such as Ubuntu, Debian or Fedora. I use Ubuntu as my primary development environment. I am comfortable with using terminal for tasks, installation and management through *apt*. I have experience in configuration of the development environment and troubleshooting issues if I run into one.

While I have not yet made direct contributions to a Linux distribution, I am interested in learning more about areas such as packaging, system tools, and how distribution components integrate together. In the future, I would like to contribute to open-source projects or Linux distributions as I continue to deepen my understanding of the Linux ecosystem.

- **Outline your experience with software packaging. This might include work with Debian packaging, RPMs, Snaps, Nix packages, Dockerfiles or otherwise.**

My experience with software packaging is mainly through working with package management tools in a Linux environment. I regularly use Ubuntu as my development environment, where I install and manage software using package managers such as *apt* and *snap*. I have also installed software directly from *.deb* packages when needed.

Through this experience, I have developed a practical understanding of how Linux package managers handle software installation, updates, and dependency management. I am comfortable using these tools to configure development environments and resolve issues related to missing dependencies or package conflicts.

I am also familiar with the concept of container-based packaging through Docker. While I have not yet had the opportunity to use Docker in production projects, I understand its purpose in creating reproducible environments and packaging applications with their dependencies.

Although my direct experience creating distribution packages such as Debian packages or RPMs is limited, I am interested in learning more about how packaging systems work internally and how software is prepared and maintained for Linux distributions.

- **According to your experience so far, how would you engineer a release pipeline for a major software project? What tools might you use for automation, and what are the critical steps in the process that should be executed in the run up to a release?**

With my experience if I were to engineer a release pipeline for a major software project which is designed to automate in a reliable and repeatable way then-

In first stage in pipeline is usually the build step, where the software is compiled/assembled. After that automatic testing with perhaps unit tests and code analysis can be performed.

Once code passes the testing stages, pipeline can move to packaging and artifact generation. Depending the project, it may be generating binaries, apks, or containers with tools such as Docker.

A release candidate is made and validated with additional testing or staging deployments. The additional testing can have dependency vulnerability checks and other security checks.

Final stage is preparing release with appropriate version in the repository, documentation and publishing it.

Critical steps in the run-up to a release can be testing, dependency checks, security checks and documentation changes. For automation tools like Gitlab CI/CD, Jenkins, Docker, etc. can be used.

Education:

- **How did you rank in your final year of high school in mathematics? Were you a top student? On what basis would you say that?**

I obtained 87/100 (A2 grade) in Mathematics and had overall percentile of 95.70 (nearly top 5%) in the Gujarat Higher Secondary Board examinations in 2020. Mathematics was one of my strongest subjects during high school, and I consistently performed well in it relative to my peers.

- **How did you rank in your final year of high school, in your home language? Were you a top student? On what basis would you say that?**

The Gujarat State Board does not include a Gujarati (my home language) examination in Grade 12. The last formal assessment was in the Grade 10 board examination, in which I scored 91/100 in Gujarati. I also studied English as a second language, scoring 94/100 in Grade 10 and 77/100 in Grade 12. Language subjects were among my stronger areas during school, and I consistently performed near the top of my class in both Gujarati and English.

- **Please state your high school graduation results or university entrance results, and explain the grading system used. For example, in the US, you might give your SAT or ACT scores. In Germany, you might give your scores out of a grading system of 1-5, with 1 being the best.**

I completed the Higher Secondary Certificate (HSC) examination in 2020 under the Gujarat Secondary and Higher Secondary Education Board (GSEB), India.

My total score was 523/650 (80.46%), corresponding to the 95.70 percentile among test takers across the state.

The GSEB evaluates students using numerical marks (generally out of 100 for theory and 50 for practical components). In addition, results are reported using a grade classification system ranging from A1 (highest), A2, B1, B2, C1, C2, D, to E, based on percentage ranges. My overall grade classification was B1, while Mathematics was graded A2.

- **Can you make a case that you are in the top 5% in your academic year, or top 1%, or even higher? If so please outline that case. Make reference where possible to standardised testing results at regional or national level, or university entrance results. Please explain any specific grading system used.**

In Gujarat Secondary & Higher Secondary Education board (GSEB) takes state wide board exams for students in class 10 and class 12 respectively.

For Gujarat Secondary School Certificate (SSC) in class 10, I took board examinations and scored 93.5% with 99.78 percentile statewide, being top 0.3% of nearly 7.9 lakh test takers in 2018.

Following for Higher Secondary Certificate (HSC) in class 12, I got 80.46% with 95.7 percentile – approximately top 5% of students across the state.

During most of my school years I was ranked first in internal school examinations, indicating consistent academic performance relative to my peers.

Both SSC and HSC examinations are standardized statewide examinations conducted by the GSEB with results reported using numerical marks and statewide percentile rankings. The percentile ranking indicate direct comparison across test takers.

- **What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for?**

In high school, I was a curious and self-motivated student who enjoyed learning beyond the classroom. I often explored parts of the syllabus early and spent time reading further simply because the topics interested me. Much of my free time went to the school library, where I read both subject-related books and novels.

Outside class, I spent time experimenting with computers and exploring programming. I had been introduced to basic programming earlier through my sister's books, and during high school I began experimenting with game development in my free time. I also participated in literature and aptitude quizzes, science fairs, math presentations, and art competitions, often earning small prizes or certificates.

My peers would likely remember me as someone curious, dependable, and approachable—someone who enjoyed learning, helped others when possible, and worked well in teams. Looking back, those experiences shaped my curiosity, work ethic, and collaborative mindset, which continue to guide me today.

- **Which university and degree did you choose? What other universities did you consider, and why did you select that one?**

I pursued a B.Tech (Bachelor's) in Computer Science and Engineering at Charotar University of Science and Technology (CHARUSAT) in Gujarat, India.

During my final years of high school, I was strongly interested in both physics and computer science. Around the COVID-19 period in 2020, I ultimately decided to pursue computer science because I enjoyed building systems and solving computational problems.

Admission to engineering programs in Gujarat is conducted through the centralized ACPC counselling system, which allocates seats based on board examination results and entrance exam rankings. I scored 80.46% in my Class 12 board examinations and achieved the 95.7 percentile. Based on my rank in the ACPC admission process, I was admitted to CHARUSAT.

Before finalizing my decision, I spoke with alumni from the university and had the opportunity to visit the campus. I had a positive impression of the academic environment,

including the computing infrastructure, laboratories, and the opportunity to interact with faculty members. These interactions reinforced my decision to pursue my degree there.

- **Overall, what was your degree result and how did that reflect on your ability? Please help us understand the grading system for your results.**

I graduated with a CGPA of 9.06/10, which corresponds to First Class with Distinction under the Indian grading system. The program evaluates students through a combination of coursework, practical assignments, and final examinations across each semester, with the CGPA representing cumulative performance across the entire degree.

Alongside theoretical coursework, the program emphasized practical experience across several core areas of computer science, including operating systems, cybersecurity, and artificial intelligence/machine learning. Many of these subjects included laboratory work and implementation-focused assignments, which helped build a strong practical foundation in system design and software development.

The curriculum also placed significant emphasis on collaborative engineering. Each semester included group-based software development projects where students were encouraged to form teams and build real-world applications from concept to implementation. These projects required us to propose ideas, justify their feasibility and usefulness, and iteratively develop them under faculty supervision.

In my case, several of these projects were supervised by the Head of the Department, who maintained particularly rigorous expectations. Project proposals were critically evaluated for originality, feasibility, and practical value, and we were expected to provide regular progress updates, prepare and present proper documentation, address technical challenges, and refine our designs throughout development. This environment helped develop disciplined engineering practices such as structured problem solving, iterative development, and clear technical communication.

- **During all of your education years, from high school to university, can you describe any achievements that were truly exceptional?**

One of my most notable academic achievements during school was my performance in the Gujarat Secondary School Certificate (SSC) examination in Grade 10. I scored 93.5% and achieved the 99.78 percentile statewide among approximately 790,000 students, placing me within roughly the top 0.3% of test takers across the state.

Throughout school I consistently ranked near the top of my class in internal examinations, particularly in analytical subjects such as mathematics and science.

Beyond formal academics, I developed a strong interest in computing and software development during high school. During the COVID-19 period I spent significant time learning game development independently, exploring areas such as procedural generation, animation systems, and visual effects while experimenting with 3D modelling.

I built several experimental modules including weapon systems, animation handling, and VFX prototypes. While many of these were exploratory rather than full games, they required

iterative experimentation and systems-oriented thinking. This period of self-directed learning played an important role in shaping my interest in computer science and software engineering.

- **What leadership roles did you take on during your education? Did you conceive of, and drive to completion, any initiatives outside of your required classwork?**

During school I served as a class representative for about a year. In that role I helped communicate between our class and the school administration, sharing announcements and raising any concerns from students when needed. I also often ended up coordinating work during group activities such as math presentations or other class projects.

In university, most courses included group software projects. In many of these teams I naturally took on a coordinating role. I usually helped define the project idea, check whether it was feasible, choose the technology stack, and divide the work across the team with weekly goals. While everyone contributed to the development, I often kept track of progress, followed up on tasks, maintained motivation and helped make sure the project stayed on schedule.

During my summer internship I worked mostly as an individual contributor but still collaborated with other developers on the team. I occasionally suggested improvements or small technical decisions during discussions, some of which were adopted in the project.

In my current role I joined as part of a graduate training program where new hires were paired and assigned a technology stack to learn. My teammate and I worked closely together to complete the training project, and our mentor later moved us into a live project earlier than most other new hires in the batch. We transitioned from Angular-based web development to Flutter and Dart for mobile development in about a month and began contributing to production work within roughly three months.

Context:

- **Outline your thoughts on the mission of Canonical. What is it about the company's purpose and goals which is most appealing to you?**

Canonical's mission to make free and open-source software practical and widely usable across modern computing infrastructure. Canonical helps open-source projects and innovation through providing reliable, secure and well-maintained platforms like Ubuntu.

The company contributes across many layers of computing from desktop systems to servers, cloud infrastructures and embedded environments. The goal of making powerful technology accessible while still maintaining high technical standards through combining open and community driven software with engineering discipline to make real-world use viable.

This very goal is I find very appealing and something I strongly respect and would be excited to contribute to.

- **What do you see as risky or mistaken in our offering, positioning or strategy?**

One potential risk or more like challenge in Canonical's positioning comes from the balance between open-source community development and the expectations of enterprise customers.

Open-source ecosystems often move quickly and encourage experimentation, while organizations that rely on these technologies in production environments tend to prioritize stability, long-term support, and predictable release cycles. Maintaining that balance can be difficult. If the platform evolves too quickly it can create friction for enterprise users, while moving too cautiously could slow innovation within the broader ecosystem.

Another challenge comes from the growing dominance of large cloud platforms. Many open-source technologies are increasingly consumed through managed services offered by cloud providers rather than being deployed directly by organizations like PostgreSQL by Amazon RDS.

While the underlying open-source projects remain widely used, the cloud platform can become the primary interface through which users interact with them. This can make it harder for companies focused on open infrastructure platforms to maintain influence over how these technologies are adopted and used.

Canonical's strategy of providing integrated tooling and support around open platforms helps address this tension, but navigating this evolving landscape will likely remain an ongoing strategic challenge.

Of course, these risks seem less like flaws in the strategy and more like natural tensions that come from operating at the intersection of open collaboration and large-scale infrastructure.

- **How should Canonical set about winning, commercially?**

For Canonical to take on the challenges in the current commercial market it has to strengthen areas where it already has advantages such as – developer collaborations, cloud infrastructure and end-to-end open source platforms.

Ubuntu is one of the most widely used OS across enterprises and public cloud. Since organisations are moving towards cloud services more and more, perhaps using Ubuntu as consistent, secure base layer for cloud provider can provide advantage.

Making guides or templates for reference architecture which can be used by key industries – hence increasing the ease of use. Also, perhaps easy migration to integrate from other platforms to Canonical and even easy to leave can win long-term trust.

Partnership with MSPs, cloud providers, etc to embedding Canonical tech in customer stacks and training & certifications to cultivate trained people who can be staffed and be trusted in deployments by customers.

- **How should Canonical amplify its impact in open source?**

Canonical has both technical credibility and influence in the ecosystem and can utilise it for investing projects upstream like Linux Kernel, Kubernetes. Canonical also can lower the barrier for ones who are willing to adopt open source but don't or can't because they struggled with complexity.

Canonical can simplify the said complexity by providing support in automation and packaging. Making powerful open source systems easy to install and operate. Especially in developer ecosystem, investing in tutorials, SDKs and community programs.

Expanding to other new domains can also amplify Canonical's impact such as edge computing and growing AI infrastructure. Especially for ensuring that these fast-growing domains stay open instead of dominated by proprietary ecosystems.

- **Why do you most want to work for Canonical?**

Reason I am particularly interested in Canonical is that my current role already sits close to the open-source ecosystem. I work on implementing a solution built on top of an open-source core project, where the company provides customized, enterprise-grade deployments for governments, NGOs, and healthcare organizations. This experience has helped me understand how open-source technologies are adapted and maintained for real-world use.

While my work currently focuses on implementation rather than contributing directly upstream, it has made me increasingly interested in participating more actively in open-source development. I would like to grow toward contributing to the technologies themselves, along with deploying and extending them.

Canonical's mission of building and maintaining widely used open-source platforms aligns strongly with that goal. The opportunity to work in an environment where engineering happens in the open and where contributions have a global impact is something I find very motivating.

I see Canonical as a place where I could deepen my systems knowledge, contribute to open-source projects more directly, and grow alongside a community that values both technical excellence and open collaboration.

- **What gives you the most satisfaction at work?**

The most satisfying part of work for me is building something that ends up being genuinely useful to others. Whether it is improving a feature for users or creating tools that make my teammates' work easier, it is rewarding to see something I helped build being used in practice.

I also enjoy debugging and problem solving. The process can sometimes be frustrating while you are in the middle of it, but the moment when the root cause becomes clear and the issue is resolved is extremely satisfying.

Another source of satisfaction is making steady progress toward a goal. Even small steps forward—fixing a bug, improving a design, or completing part of a feature—add up over time. Being able to look back and see how much the system or project has evolved because of those incremental improvements is very motivating for me.

- **What would you most want to change about Canonical?**

From the outside, I recognize that my understanding of Canonical is limited to public information and community discussions, so I would approach this cautiously. Overall, the company's mission and technical direction appear clear and well aligned with the open-source ecosystem.

One area that might offer further opportunity is lowering the barrier for new developers to explore and contribute to Canonical's ecosystem. Many of the technologies Canonical works on—cloud infrastructure, containers, and large distributed systems—are powerful but can also appear complex to newcomers.

Continuing to invest in approachable documentation, clear development environments, and simple ways for developers to experiment with Canonical technologies could help attract even more contributors and users. Expanding the path from “curious developer” to “active contributor” would further strengthen the surrounding open-source community.

That said, this is more of an external perspective than a firm criticism, and I would expect that understanding the internal context would provide a more complete view.

- **What gets you most excited about this role?**

What excites me most about this role is the opportunity to work directly on open-source systems that are used widely across the world. Software like Ubuntu forms part of the infrastructure that developers, companies, and researchers rely on every day, and contributing to something with that level of reach is very motivating to me.

I have always been interested in the ideas behind Linux and open source—the collaboration, transparency, and the goal of building technology that anyone can use and improve. Being able to work on systems that reflect those principles would be both meaningful and technically exciting.

Earlier in my career I assumed that contributing to projects at this scale would only be possible after gaining several more years of experience. The possibility of contributing as part of a junior role is exciting because it provides a chance to learn from experienced engineers while developing skills in systems and open-source development much earlier in my career.